FACIAL EXPRESSION EMOTION RECOGNITION

Muhammed Arif Mohd Ali Jinnah, Shuhail Azri bin Md Hanif, Imran bin Zulkiflee, Raja Muhammad Darwisy bin Raja Ahmad Al'Konee

***Abstract* — Emotion recognition through facial expressions has gained popularity in recent years with the advent of deep learning techniques. The ability to automatically identify emotions through facial features has potential applications in industries such as marketing, mental health, customer service, and law enforcement. However, recognizing emotions from facial expressions remains challenging due to the diversity of human emotions and the influence of cultural, personality, and individual factors. This study evaluates the feasibility of using various deep learning models, specifically CNN architectures, for emotion recognition through facial expressions. We propose different classifier parameters and investigate the impact of data preprocessing and augmentation on the model's performance and prediction accuracy. The findings of this study will contribute to the advancement of the field and inform the development of more accurate and efficient models for emotion recognition in real-world applications.**

Keywords — Emotion recognition, deep learning, CNN, DenseNet201, EfficientNetB0, ShuffleNetV2, MobileNetV2, facial expression classification

1. INTRODUCTION

With a substantial section of the tech-savvy public being familiar with this technology, the focus around computer vision has undoubtedly reached new heights. Deep learning, a type of machine learning, is increasingly being used in facial recognition applications across a variety of sectors. A subclass of face recognition software involves identifying an individual's emotion from their facial characteristics.

The capacity to identify and analyse the emotions that people express in photos or films is known as emotion recognition. Emotional labels can be added to images or videos or facial expressions can be coded manually, both of which are classic methods for identifying emotions. These methods require a lot of time and are subject to bias. Automating emotion recognition using machine learning algorithms, particularly deep learning methods, has grown in popularity in recent years. These algorithms have shown good performance in a variety of tests for emotion identification, and they have the potential to greatly improve the efficiency and accuracy of emotion recognition systems.

Facial emotion analysis has potential uses in marketing and advertising, among other fields. Businesses might utilise emotion analysis to better understand how customers are responding to their products or campaigns and adjust their marketing tactics as necessary. In addition, it might be used to raise awareness of mental health issues by identifying and tracking changes in mood and emotional state, which could be helpful for identifying and treating mental health concerns. By recognising and responding to consumers' emotional needs and concerns in real-time, customer service domains may also utilise this approach to enhance their interactions with customers. Last but not least, law enforcement might use emotion recognition to detect and monitor changes in emotional state during police interrogations or other legal procedures.

Despite these developments in recent years, it is still challenging to identify facial expressions of emotion due to the complexity and range of human emotions. Humans commonly employ actions like facial expressions, body language, and other environmental cues to convey their emotions. In addition, a wide range of factors, including culture, personality, and personal traits, can influence how emotions are portrayed. To build an effective and accurate facial expression emotion detection system, a combination of machine learning methodologies, human expertise, and knowledge of the underlying psychological and social variables that influence emotions is required.

In this study, we investigate various deep learning models, chiefly convolutional neural network (CNN) architectures, in their feasibility of solving the problem of emotion recognition through facial expressions. In order to improve performance, different parameters in relation to the dataset are suggested. The effectiveness of the model and the precision of the predictions are also examined in relation to data preprocessing and augmentation done to the dataset.

2. BACKGROUND / RELATED WORK

2.1. DenseNet-201

DenseNet-201 is a deep learning architecture for classifying images [1]. The number "201" in the name alludes to the network's 201 levels. Because there are more layers in DenseNet-201 than in other CNNs, the data can be represented in more intricate ways.

Dense connections, rather than only those between directly connected layers, form the foundation of the DenseNet-201 architecture, which allows each layer to receive input from all layers that came before it. This enables better gradients and information flow during

training, which makes it simpler for the network to pick up complicated patterns in the data.

In comparison to models like VGG and ResNet, Dense Convolutional Network (DenseNet) has dense connections. Direct connections from any layer to all succeeding layers distinguish the DenseNet model from other CNNs and can further improve the information flow between layers. As a result, DenseNet may effectively minimise some parameters, improve feature map propagation, and solve the gradient vanishing problem. According to new research, on the ImageNet classification challenge, DenseNet performed best for the detection of multiple sclerosis and surpassed other pretrained models, as can be seen in Figure 1 [2].

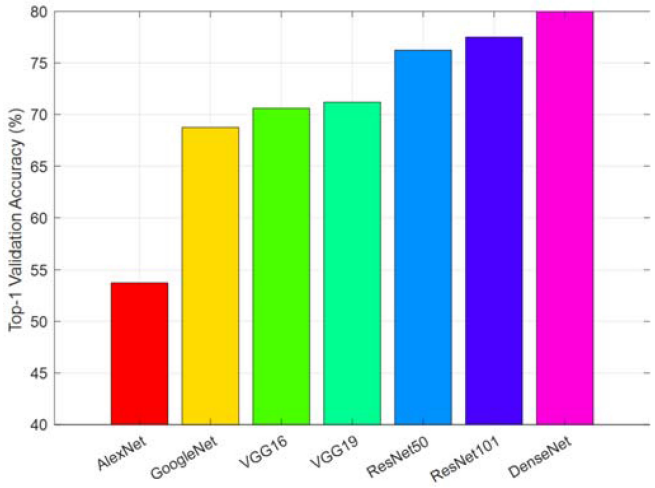


Figure 1 : Comparison of pretrained models [2].

2.2. EfficientNet-B0

CNN architecture EfficientNet-B0 was created by Google AI with the goal of being more effective and efficient than current designs for image categorization tasks. EfficientNet-B0 is among the models in the EfficientNet family, which is intended to be more effective and efficient than current designs. In particular, computer vision and image processing tasks including image classification, object identification and visual recognition have all made use of EfficientNet-B0.

Training a machine learning model to recognise and analyse the emotions communicated by facial expressions in photos or videos is the problem of visual emotion recognition using EfficientNet-B0. The EfficientNet-B0 model is the smallest in the family and is intended to be lightweight, making it more suited for running on less powerful hardware, such mobile or normal computer devices. It requires less processing since it has fewer parameters than the EfficientNet family's bigger models [3].

EfficientNet has been utilised as a foundation model for image classification tasks, where it has been adjusted to reach excellent accuracy on various datasets. The model's modest size makes it suited for use on devices with limited resources. EfficientNet has been adapted to predict and classify lymph node metastasis in breast cancer [4]. The study utilised EfficientNetB3 which achieved an accuracy of 97% and an AUC value of 99%. The result may offer an effective, trustworthy, and affordable option to medical institutions in crucial locations.

2.3. ShuffleNetV2

ShuffleNetV2 is a lightweight deep neural network architecture, with emphasis placed on effective mobile computing. It was released in 2018 as an enhancement to the initial ShuffleNet architecture. Some of the drawbacks of the original ShuffleNet are addressed by ShuffleNetV2, which has superior compute and memory efficiency, enhanced accuracy, and increased scalability. For mobile devices with 10-15 MFLOP (mega floating-Point operations per second) computer capacity, ShuffleNet is a CNN architecture that is computationally efficient [5].

It accomplishes this by providing a brand-new channel shuffling procedure, which helps to simplify computation and enhance feature reusability. It utilises pointwise group convolutions and channel shuffling to achieve improved computational efficiency, reducing the number of parameters and computations needed without sacrificing accuracy. The architecture also includes a residual connection structure and uses depth-wise separable convolutions to further reduce computation time.

ShuffleNetV2's contribution is to offer the following four design principles for effective networks:

- Minimising the memory access cost in the convolution operation by maintaining an equal number of input and output channels for the convolutional layer.
- Lowering the usage of group convolution to reduce the cost of memory access.
- Minimising element-level activities to accelerate network operation.
- Minimising branch structure during network design to improve the capability of parallel operation.

According to their experimental findings, ShuffleNetV2 is more accurate than MobileNetV2 and Xception for the same level of model complexity.

Using ShuffleNetV2 for vegetation density classification, the batch size 32 results using epoch 75 achieved 91.1% results, while the total number of batch sizes 64 using epoch 75 obtained 91% results [6]. A hand gesture recognition system utilising ShuffleNetV2 can categorise 7 gestures with a promising accuracy of 98.52% while analysing the patterns of various gesture RDM images

[7]. In a garbage classification system, GCNet, parallel mixed attention mechanism (PMAM), the usage of novel activation functions, and transfer learning are three enhancements made to ShuffleNet v2. The experimental findings demonstrate that the average GCNet accuracy on the self-built dataset is 97.9%, with just 1.3M model parameters. Therefore, GCNet only needs 0.88 seconds to finish classifying and gathering a single object, once again highlighting the lightweight nature of ShuffleNetV2 [8].

2.4. MobileNetV2

Google has created a series of deep neural network designs called MobileNet that are intended for use in embedded and mobile vision applications. The main goal of MobileNet is to create a neural network architecture that is suitable for these kinds of applications while still being efficient and effective.

MobileNetV1 reduces the computational cost of the model while retaining acceptable accuracy by combining depthwise separable convolution and 1x1 pointwise convolutional layers [9]. The "depthwise convolution" method of convolutional layers, which is used by MobileNetV1, provides for a reduction in computation and parameter requirements.

By adding a new construction block dubbed the "inverted residual block" [10], which further lowers the computing cost while enhancing accuracy, MobileNetV2 improves on the architecture of MobileNetV1. This construction element makes better use of the model's processing resources by combining depthwise separable convolution and linear bottleneck layers.

Due to their great performance and minimal computing needs, MobileNet models have gained popularity in a variety of computer vision applications, particularly in object recognition and classification. They have been utilised in several real-world applications, including augmented reality, object identification, facial recognition, and picture categorization.

3. APPROACH

In this study, several CNN architectures will be explored in solving the emotion recognition through facial expressions problem. Possible modifications of these architectures as well as their parameters in context to the dataset will be proposed in achieving better performance. Data preprocessing and augmentation applied to the dataset will be investigated in terms of model performance and prediction accuracy. The goal of this study is to identify the most effective deep learning architecture for emotion recognition through facial expressions. To achieve this, we will compare several convolutional neural network (CNN) architectures, namely DenseNet-201, EfficientNet-B0, ShuffleNetV2 and MobileNetV2. The CNNs will be trained on the publicly available Facial Expression Recognition 2013 (FER-2013) dataset.

In addition to evaluating the baseline CNN architectures, we will also explore modifications to the architecture and its parameters to improve performance. This will involve using techniques such as transfer learning and data augmentation. To ensure a fair comparison, all models will be trained using the same training and validation splits. We will use a combination of evaluation metrics to analyse the performance of the models. Overall, this study will provide valuable insights into the use of deep learning for emotion recognition through facial expressions.

3.1. DenseNet-201

The process of categorising a person's emotional state based on their facial expressions, body language, speech, or other indicators can be accomplished with DenseNet-201. When classifying emotions into predetermined categories, such as happy, sad, furious, and so forth, the DenseNet-201 architecture can be utilised to extract characteristics from the input image.

Through practice on a sizable collection of emotion-labelled image examples, the features can be learned. The network is exposed to input photos and the related emotional labels during training. The network then modifies its weights to reduce the difference between the expected and actual emotions' classification errors. By forward propagating the image through the network and using the output to create a forecast, the DenseNet-201 model may be used to make predictions on new, unseen images after being trained. In general, the deep structure and the dense connections between the layers of DenseNet-201 have shown to be an efficient architecture for picture classification tasks, including emotion recognition.

3.2. EfficientNet-B0

One of the methods used in this Facial Expression Emotion Recognition study is using EfficientNet-B0. Being the smallest model in the EfficientNet family, it is designed to be more efficient and effective than existing architectures for image classification tasks. In Figure 2, it is shown that the smaller the model of EfficientNet, the lower the number of parameters it is required. It may also indicate that a higher version of EfficientNet is more computationally expensive as it requires more parameters and takes longer time to run an epoch.

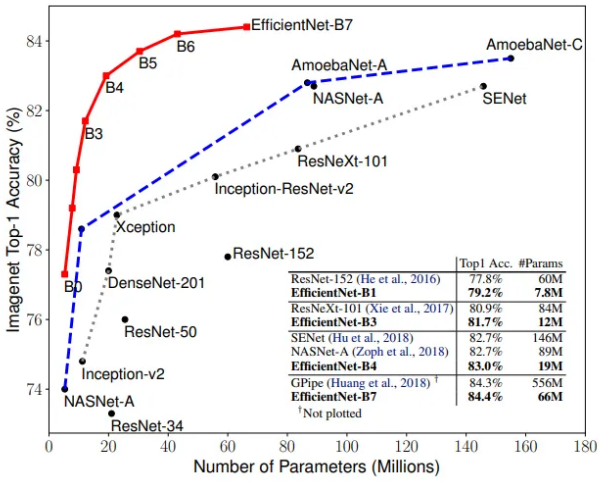


Figure 2 : Comparison of architectures [3].

EfficientNet-B0 employs a compound scaling technique that modifies the model's width, depth, and resolution in a consistent and logical way. The model can attain great accuracy while using fewer parameters by modifying these parameters. Better accuracy can be achieved by scaling CNN which is the architecture to create EfficientNet but no in-depth analysis of the scaling procedure has been conducted [11]. The author also mentioned that by simply expanding the CNN depth or width or by utilising a higher input picture resolution, it required an iterative human tweaking procedure. A compound scaling method was proposed that uses a fixed set of coefficients to uniformly scale width, depth, and resolution, which was named as EfficientNet B0 [12].

In this study, the EfficientNet-B0 model architecture, which trains the FER2013 dataset on pre-trained weights of ImageNet is used. When the weight parameter for a deep learning model is set to "imagenet," pre-trained weights from the ImageNet dataset will be used as the model's starting weights. With over 14 million photos [13], the ImageNet dataset is a sizable collection of images for image classification applications. In order to gain valuable features that may be utilised for image classification tasks, several well-known deep learning architectures, such as EfficientNet, are pre-trained on the ImageNet dataset.

3.3. ShuffleNetV2

ShuffleNetV2 is a sophisticated net design [6]. Although it may be said that each block's half of the channel is reused, the nonlinearity and expressibility are somewhat diminished by the fact that each block's half of the channel instantly joins the following block. With 24 channel filters measuring 3x3 and a step value (stride) of 2, the first convolutional layer (Conv1) is a sort of CNN layer. The following stages are stage 2, stage 3, and stage 4, each of which consists of two stages of shuffling with their corresponding strides, 2 and 1 correspondingly, as seen in Figure 3. The second shuffling stage of stages 2, 3, and 4 is repeated, up to 2, 4, and 2 times, respectively.

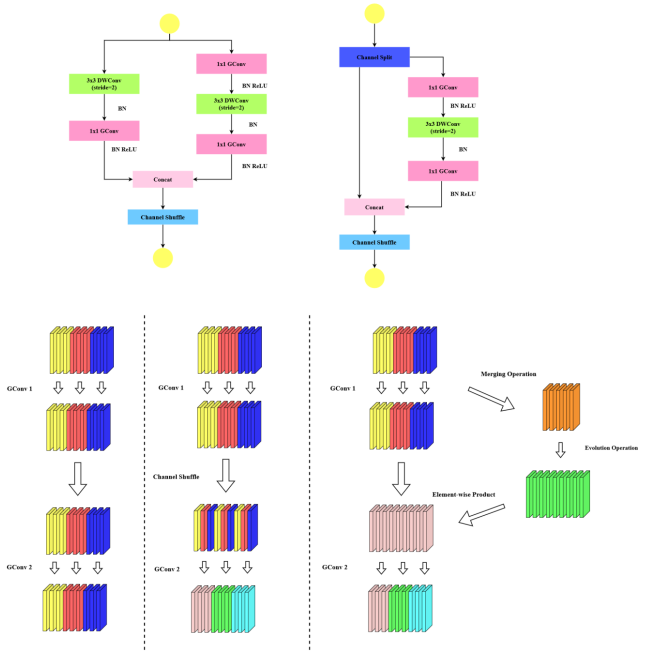


Figure 3 : ShuffleNetV2 architecture [6].

The Shuffle channel's performance process begins with a convolution group, and each group receives convolutional results from another group. As a consequence, each group on the Shuffle channel is made up of a mixture of channels from other groups, which strengthens the correlation.

To divide the following convolution stage into several combinations, the characteristics produced by the preceding group can be separated into subgroups. The shuffle unit executes it effectively and elegantly. In this study, the ShuffleNetV2 model architecture is built based on [14], which is then trained on the FER-2013 dataset, with no pretrained weights.

3.4. MobileNetV2

The model is built using a set of inverted residual blocks, which make up MobileNetV2's overall architecture, as seen in Figure 4. A mixture of depthwise separable convolution and linear bottleneck layers makes up each inverted residual block. Each input channel is given its own convolutional filter by the depthwise separable convolution layer, while the linear bottleneck layer minimises the number of output channels.

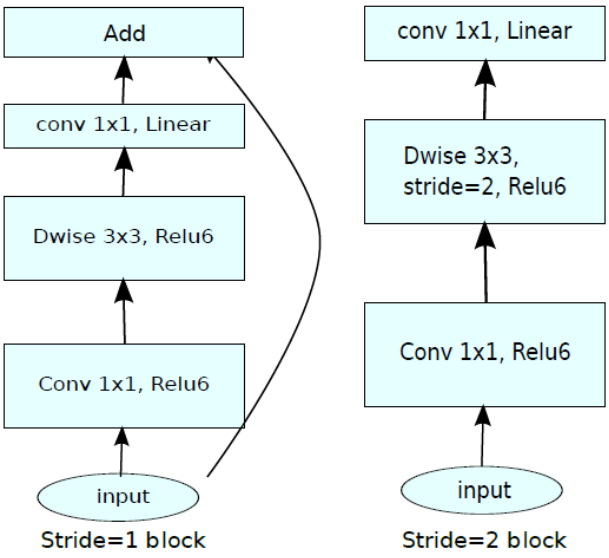


Figure 4 : MobileNetV2 Architecture [10].

To decrease the number of input channels, the input is first processed through a 1x1 pointwise convolution. The output of this layer is then fed through a depthwise convolution, which gives each input channel a different convolutional filter. The depthwise convolution's output is then subjected to a final 1x1 pointwise convolution, which multiplies the output channels and combines the depthwise convolution's features. The network may learn residual mappings, which may be more effective and simpler to optimise than direct mappings, by adding the output of each inverted residual block to the input, a process known as the "shortcut connection."

The "expansion factor" is a hyperparameter that may be used to regulate how many output channels are generated for each inverted residual block. After depthwise convolution, the 1x1 pointwise convolution has a number of output channels that are controlled by the expansion factor. The trade-off between computational expense and precision can be managed in this way. Additionally, the "linear bottlenecks" function of MobileNetV2 is utilised to cut down on the number of output channels in the 1x1 pointwise convolution layer of the inverted residual block. This contributes to further lowering the model's computational expense without significantly lowering accuracy. A number of datasets, like the ImageNet dataset, which has over 1.2 million photos labelled with 1000 classes, may be used to train MobileNetV2. The model may be used to categorise photos or extract characteristics from them after being trained.

4. EXPERIMENT

4.1. Dataset

In this study, the FER-2013 dataset is used for the training and validation of the proposed models. The dataset is publicly available and is retrieved from Kaggle [15]. It was prepared by Pierre-Luc Carrier and Aaron Courville, as part of an ongoing research project [16].

The data consists of 35887 grayscale images of faces of size 48x48 pixels. The training set consists of 28709 examples while the validation set contains 3589 examples. A testing set is also provided containing another 3589 examples, which will be used for performance metrics. The emotions portrayed in the images are labelled as: 0 - Angry, 1 - Disgust, 2 - Fear, 3 - Happy, 4 - Sad, 5 - Surprise, 6 - Neutral. The faces have been automatically registered so that the face is more or less centred and occupies about the same amount of space in each image. The distribution of the dataset is shown in Figure 5 below.

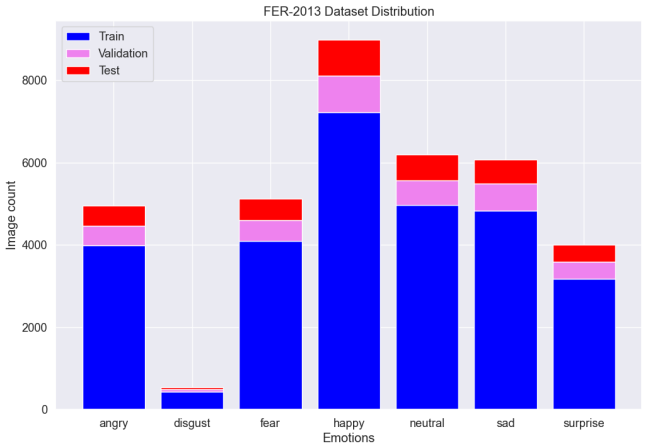


Figure 5 : FER-2013 dataset distribution.

Originally the dataset is imbalanced as represented by two minority classes, ‘disgust’ and ‘surprise’, as well as a majority class, ‘happy’.

To deal with the imbalanced dataset, we applied data augmentation to oversample the minority classes, ‘disgust’ and ‘surprise’. Data augmentation is a strategy that includes producing more samples from the current data to artificially increase the size of the dataset. Both the model's generalisation and its performance on tasks requiring emotion detection may benefit from this. Data augmentation works by doing different transformations to the data that is obtained. This is carried out to avoid overfitting, which is a frequent issue when deep learning models are trained.

The augmentation processes include Gaussian blur, width and height shift, horizontal flipping and rescaling. These processes are done using the Albumentations package [17]. Meanwhile, the majority class, ‘happy’, was randomly undersampled. The size of the class ‘surprise’, after being subject to oversampling, was also readjusted through random undersampling. The resulting balanced distribution is shown in Figure 6 below.

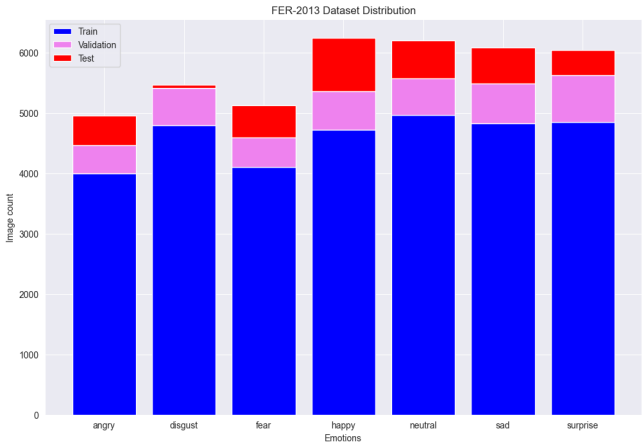


Figure 6 : Balanced dataset after image augmentation oversampling and random undersampling.

As can be seen from the figure above, we only applied the augmentation to the train and validation sets. ImageDataGenerator was also utilised to perform data augmentation for the entire dataset before the data is fed into the models. The ImageDataGenerator class in Keras offers a practical method for adding data in the dataset. When the data is fed into the model using this class, a set of data augmentation parameters may be specified, and it will automatically be applied to the data. This makes it simple to test out several data augmentation techniques and choose the one that is most effective for a particular use case.

4.2. Classifier Parameters

4.2.1. Classifier Parameter 1

The first parameter inputs are put through a process called global average pooling. Using this procedure, which creates a tensor with a smaller spatial dimension by averaging all values in each feature map of the input tensor, the network is able to recognise the most important features in the input. A 256 unit dense layer with ReLU activation function is added after that. The layer additionally has an L2 regularisation term with a strength of 0.01. This assists in preventing overfitting by penalising the loss function during training. At that point, it adds a 0.3 dropout layer.

By randomly setting a fraction of the input units to 0 during training, the regularisation technique known as dropout helps prevent overfitting. This line produces another dense layer with 1024 units, a ReLU activation function, and an L2 regularisation term with a strength of 0.01. then creates a second dropout layer with a 0.5 dropout rate. creates a new dense layer with 512 units, a ReLU activation function, an L2 regularisation term, and a strength of 0.01 in the regularisation term. With a rate of 0.5, this line adds a further dropout layer. It then uses 7 units and a softmax activation function to construct the final dense layer. The input is categorised into one of the 7 categories by this layer. The layer's name is currently set to "classification."

4.2.2. Classifier Parameter 2

Several layers are used to define the architecture of the model. In the modified layer structure for this problem, the model is sequential which means the layers are added in a linear way one after the other. In this sequential model, the model is initialised as a sequential model and a pre-trained base model is added to the sequential model in the following layer which is the models used. The dropout layer with a dropout rate of 0.5 is added in the next layer. In order to avoid overfitting, this layer is designed to randomly remove a set proportion of neurons during training. The output from the previous layer is flattened in the next layer. The two layers after that add a dense layer with 32 units and a He_uniform initialization, as well as a batch normalisation layer standardising inputs to a layer to enable better and quicker learning. The addition of a batch normalisation layer, an activation layer using ReLU activation, and a dropout layer with a dropout rate of 0.5 are repeated in the next three layers. Lastly, the softmax activation function and a dense layer with 7 units which are the 7 emotions in this project are included in the last layer.

4.3. Evaluation Metrics

The performance evaluation measures for each model, namely Accuracy and Confusion Matrix are described in this section. As such, the expected discussion of the results from the experimentation will detail the metrics mentioned in the subsections below while including plots portraying the metrics and the distribution of predictions against the actual emotion classes.

4.3.1. Accuracy

One assessment measure that is frequently used in image classification issues is accuracy. The most simple performance statistic is accuracy, which is defined as the ratio of properly predicted picture classes to the total number of photos. A percentage is typically used to represent classification accuracy, with larger numbers signifying greater performance and easy interpretability [18].

4.3.2. AUC-ROC Score

AUC-ROC is used to assess how well a classification model performs while trying to predict classes [18]. It measures the model's capacity to differentiate between the classes and is an effective statistic to employ when the classes are unbalanced. The ROC curve, on which the AUC-ROC is based, shows the true positive rate versus the false positive rate on the y-axis. The area under the ROC curve is known

as the AUC-ROC. A score of 0.5 means the model performs no better than random, whereas a value of 1 shows perfect discrimination.

4.3.2. Confusion Matrix

The confusion matrix is also a commonly used evaluation metric for assessing a model's performance in image classification problems. It displays the proportion of accurate and inaccurate predictions the model produced for each dataset class. The actual class labels are listed on one axis of the confusion matrix, while the predicted class labels are listed on the other. The entries in the matrix show how many test set instances were assigned to one class while they actually belonged to another. In this study, the confusion matrix is displayed as a heatmap which makes it simpler to identify trends and patterns in the model's performance.

4.4. DenseNet-201

4.4.1. Training Imbalanced Dataset

For the first parameter, the accuracy of the training set is 72% after the architecture layers have been fitted, whereas the accuracy of the validation set is 63%. The training and validation accuracy are both steadily improving over time, as seen in Figure 7. This indicates that the model is successfully learning. However, from the loss graph, the model is overfitting and cannot generalise on the new data as the gap between validation and training loss is big with validation being higher than training.

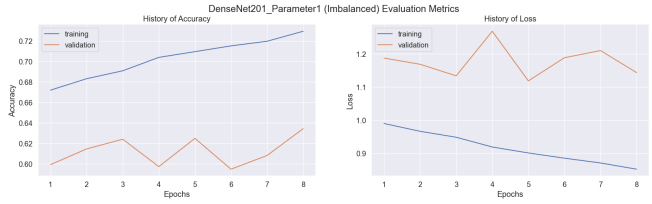


Figure 7 : Training and validation using DenseNet-201, Parameter 1 on Imbalanced dataset

For the second parameter set, the accuracy of the training set is 58% after the architecture layers have been fitted, whereas the accuracy of the validation set is 54%. The training and validation accuracy are also both steadily improving over time, as seen in Figure 8. This indicates that the model is also successfully learning and is not overfitting. The model's loss for both training and testing is continuously dropping over time, similar to the accuracy graph, which is a strong indication that the model is improving over time and doing better on the validation set.

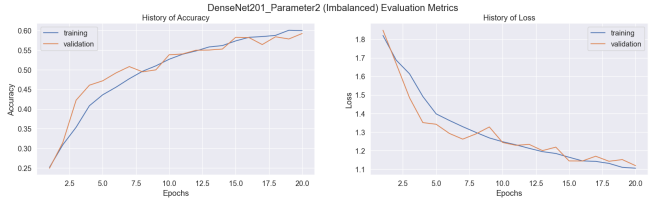


Figure 8 : Training and validation using DenseNet-201, Parameter 2 on Imbalanced dataset

4.4.2. Training Balanced Dataset

Figure 9 shows that, after the architectural layers have been fitted, the training set's accuracy is 75% while the validation set's accuracy is 65% for the first classifier parameter set. As shown in the figure, both training and validation accuracy are increasing over time. The model is learning successfully, but it is overfitting and unable to generalise on the new data because of the large disparity between validation and training loss, with validation being larger than training. This is similar to training on the imbalance dataset.

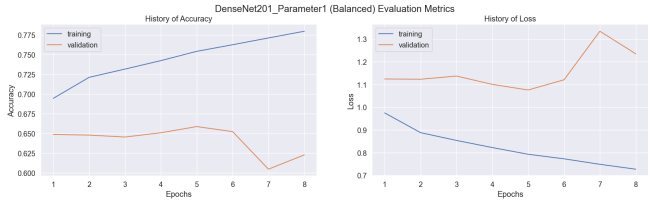


Figure 9 : Training and validation using DenseNet-201, Parameter 1 on Balanced dataset

After the architecture layers have been fitted, the accuracy of the training set for the second parameter set is 64%, whilst the accuracy of the validation set is 63%. As shown in Figure 10, both the training and validation accuracy have been progressively increasing over time. This shows that the model is not overfitting and is successfully learning as well. Similar to the accuracy graph, the model's loss for both training and testing is steadily decreasing over time. This is a very strong signal that the model is improving over time and performing better on the validation set.

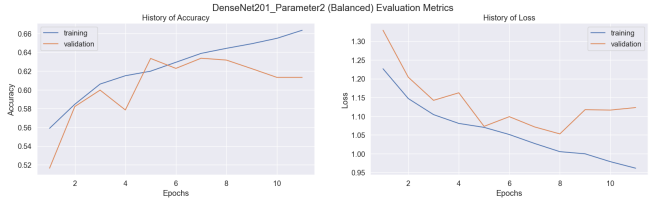


Figure 10 : Training and validation using DenseNet-201, Parameter 2 on Balanced dataset

4.4.3. AUC-ROC and Confusion Matrix

Using DenseNet-201, after training on the imbalanced and balanced datasets, the AUC-ROC plots and scores as well as the confusion matrix obtained are exactly similar in terms of dataset distribution, with differences only noticed between classifier parameters.

Upon implementing the data to both architecture parameters, the AUC-ROC score achieved by this model is 89% and 88%, on parameter sets 1 and 2 respectively, as seen in Figure 11 and Figure 12.

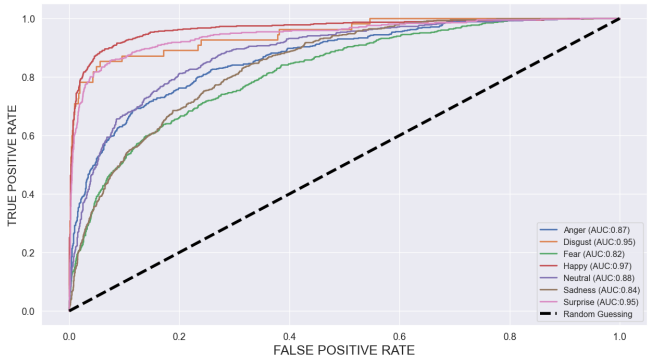


Figure 11 : ROC curve of DenseNet-201, Parameter 1

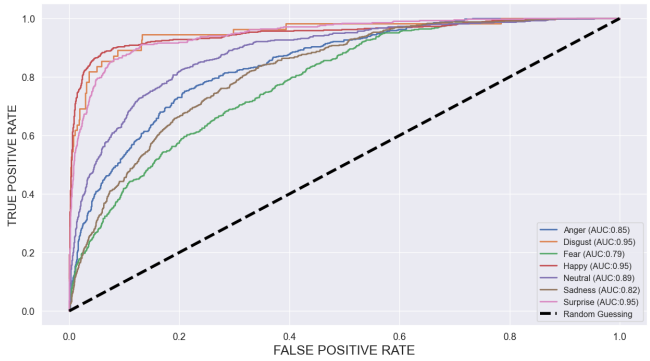


Figure 12 : ROC curve of DenseNet-201, Parameter 2

Figure 13 shows the confusion matrix for parameter 1 for balanced and imbalanced with an overall accuracy of the classifier, calculated as the number of correct predictions divided by the total number of predictions. All of the classes are predicted well and not far from the actual value. We looked at the confusion matrix and metrics for each emotion in order to better comprehend and evaluate the models. Results of the dataset's facial expression recognition for seven classes are shown in the DenseNet-201 confusion matrix. The result of confusion shows the predicted 7 classes. It shows that the class happy with the value of 725 is the highest compared to the other classes.

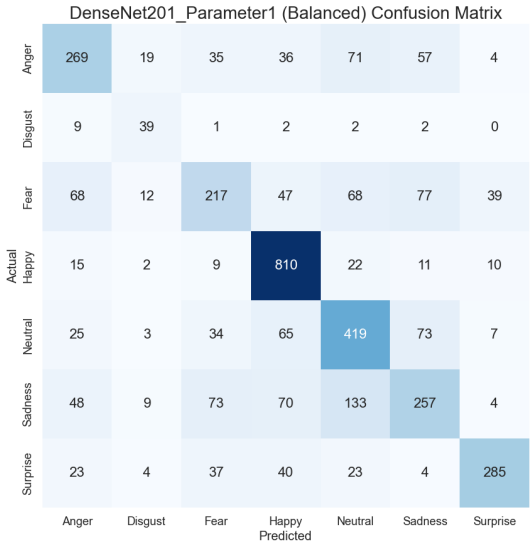


Figure 13 : Confusion matrix of DenseNet-201, Parameter 1

Figure 14 shows the confusion matrix for parameter 2 for balanced and imbalanced DenseNet-201. Most of the classes are predicted well and not far from the actual value. We looked at the confusion matrix and metrics for each emotion in order to better comprehend and evaluate the models. Results of the dataset's face expression recognition for seven classes are shown in the DenseNet-201 Confusion matrix. Most of the actual class for disgust samples falsely predicted anger and other classes are predicted well . This shows that in parameter 1 the confusion matrix is better compared to parameter 2 using DenseNet-201.

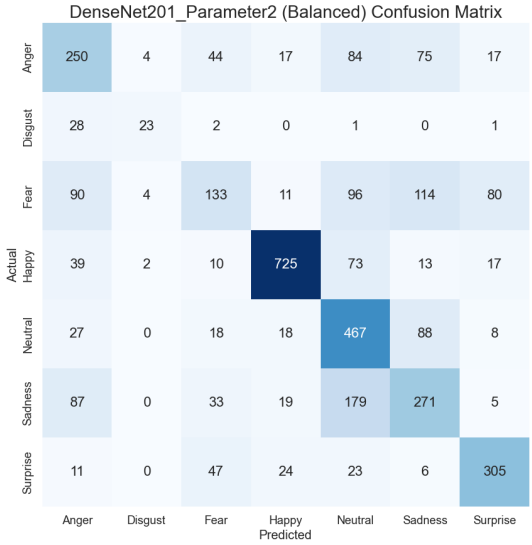


Figure 14 : Confusion matrix of DenseNet-201, Parameter 2

4.5. EfficientNet-B0

4.5.1. Training Imbalanced Dataset

After fitting the first architecture layers onto the imbalanced training dataset, the results are shown as in Figure 15. It is resulting in the accuracy of this model on the training data is 34.2% and the accuracy of this model on the validation dataset is 24%. Compared to DenseNet-201, the accuracy achieved by EfficientNet-B0 has significantly dropped even when using the same parameters. The history of loss on validation is fluctuated in seventh epochs and may be caused by the sequential building architecture. A spike on the loss graph which may be caused by class distribution being highly imbalanced makes the model struggle to learn from the underrepresented classes.

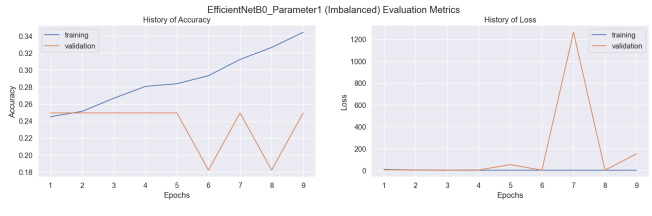


Figure 15 : Training and validation using EfficientNet-B0, Parameter 1 on Imbalanced Dataset

While using the second architecture parameter, The accuracy of this model on the training dataset increases to 25.2% but the validation stays on 24.8%. Loss for both training and validation drops steadily which indicates that the model is learning and increasing its performance on the training data.



Figure 16 : Training and validation using EfficientNet-B0, Parameter 2 on Imbalanced Dataset

Upon implementing the data to both architecture parameters, the AUC-ROC score achieved by this model is only 47%.

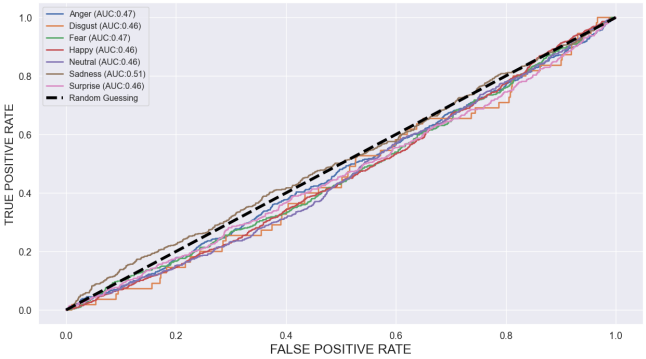


Figure 17 : ROC curve of EfficientNet-B0 on Imbalanced dataset

4.5.2. Training Balanced Dataset

As fitting the first architecture layers onto the balanced training dataset, the results are shown as in Figure 18.The accuracy of this model on the training dataset is 35% and the accuracy of this model on the validation dataset is 16%. A spike on the validation loss graph may indicate overfitting, where the model is memorising the training data instead of generalising to new data. In addition to that, it is clear that the training graph is moving further away from the validation data score.

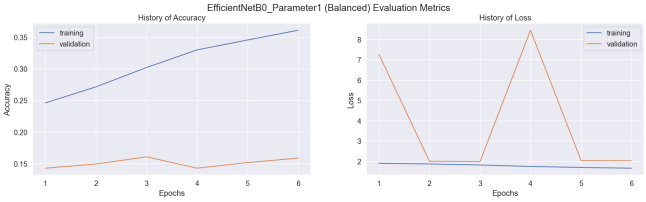


Figure 18 : Training and validation using EfficientNet-B0, Parameter 1 on Balanced Dataset

As fitting the second architecture layers onto the balanced training dataset, the results are shown as in Figure 19. The accuracy of this model on the training dataset is 15% and the accuracy of this model on the validation dataset is 16%. Loss for training drops steadily which indicate that the model is learning and increasing its performance on the training data.

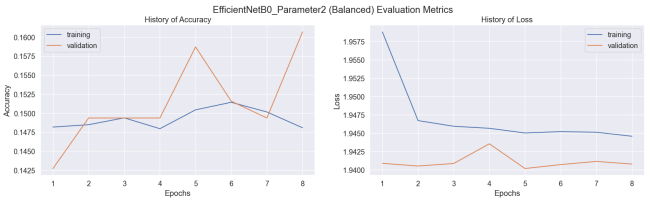


Figure 19 : Training and validation using EfficientNet-B0, Parameter 2 on Balanced Dataset

While the score for AUC-ROC is better for a balanced dataset for EfficientNet-B0 which is 51%, it is still far from the DenseNet-201 score and tells that DenseNet-201 performs better than EfficientNet-B0.

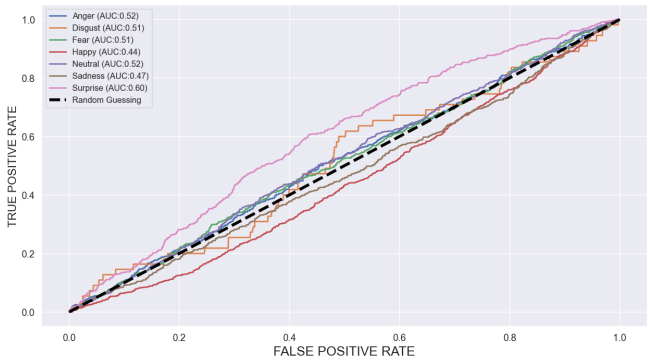


Figure 20 : ROC curve of EfficientNet-B0 on Balanced dataset

From the confusion matrix shown in the figure below, it is explained why the accuracy is low because it may only predict one class from the dataset even when the dominant class is happy. Hence, confusion matrix is not reliable for the evaluation of EfficientNetB0.

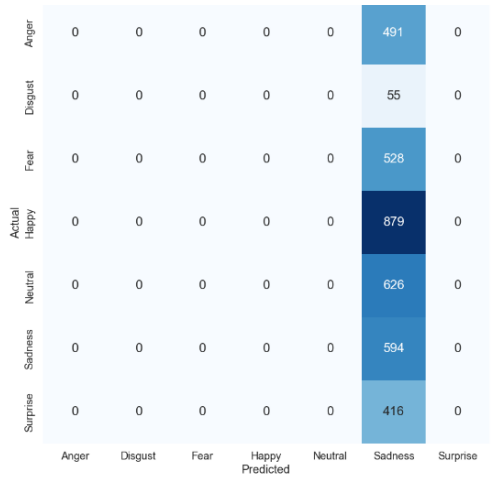


Figure 21 : Confusion matrix of EfficientNet-B0

4.6. ShuffleNetV2

4.6.1. Training Imbalanced Dataset

When training ShuffleNetV2 with the first classifier set applied on the imbalanced dataset, the results are shown as in Figure 22. The accuracy of the model on the training data is 25.15% while the validation accuracy is 24.94%. In comparison to DenseNet-201, the accuracy achieved by ShuffleNetV2 is significantly lower, although similar parameters are applied. A spike on the loss graph correlates to the imbalanced class distribution, and as a result the model might have struggled to learn from the underrepresented classes.

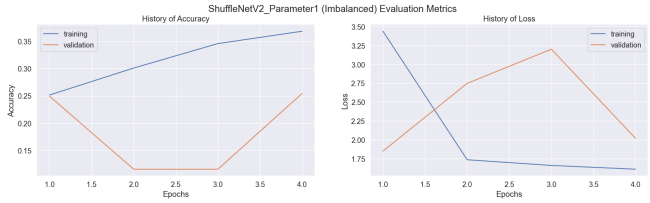


Figure 22 : Training and validation using ShuffleNetV2, Parameter 1 on Imbalanced Dataset

Meanwhile, applying the second classifier parameter, the accuracy of the model on the training data drops to 20.4%. The validation accuracy is similar to the first parameter set, at 24.94%. With an imbalanced class distribution, the spike in the loss graph as seen in Figure 23 suggests the model might have struggled to learn from the underrepresented classes, similar to the results from the first parameter set.

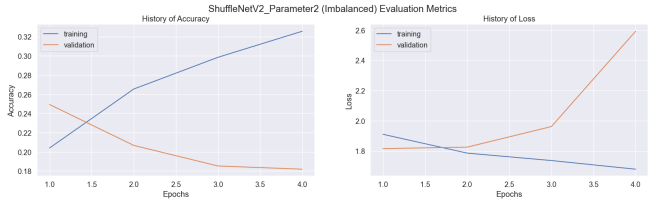


Figure 23 : Training and validation using ShuffleNetV2, Parameter 2 on Imbalanced Dataset

4.6.2. Training Balanced Dataset

Figure 24 portrays the results from implementing ShuffleNetV2 on the balanced dataset with the first parameter set. The accuracy of the model on the training data increases to 41.23%. The validation accuracy, however, drops to 15.97%. The sharp drop on the validation loss graph may indicate overfitting, where the model is memorising the training data instead of generalising to new data. This can be seen that the training accuracy is much different than on the validation set.

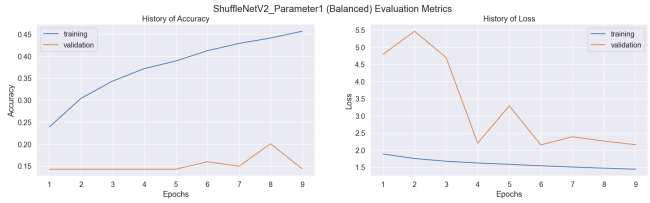


Figure 24 : Training and validation using ShuffleNetV2, Parameter 1 on Balanced Dataset

Applying the second parameter set leads to a training accuracy of 19.23% and a validation accuracy of 18.82%, as seen in Figure 25.

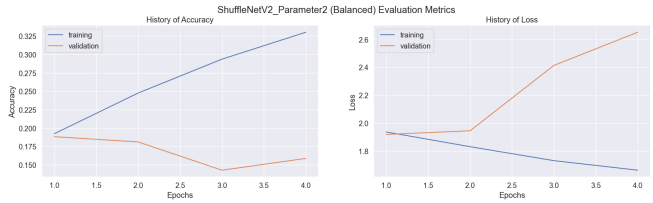


Figure 25 : Training and validation using ShuffleNetV2, Parameter 2 on Balanced Dataset

Even with a balanced class distribution, the model with this set of classifier parameters seems to struggle with learning.

4.6.3. AUC-ROC and Confusion Matrix

Upon implementing the data to both architecture parameters, the AUC-ROC score achieved by ShuffleNetV2 is 61.58% and 58.13%, on parameter sets 1 and 2 respectively, as seen in Figure 26 and Figure 28, which indicates the model's low class interpretability.

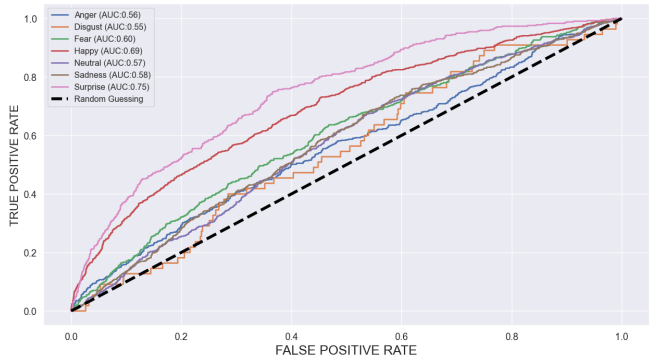


Figure 26 : ROC curve of ShuffleNetV2, Parameter 1

From the confusion matrix shown in Figure 27, it can be fathomed as to why the accuracy is low because it may only predict one class from the dataset, which is the majority class, 'happy'. Likewise, in Figure 29, the confusion matrix only predicts the classes, 'sadness' and 'surprise'. Hence, the confusion matrix is also not reliable for the evaluation of ShuffleNetV2.

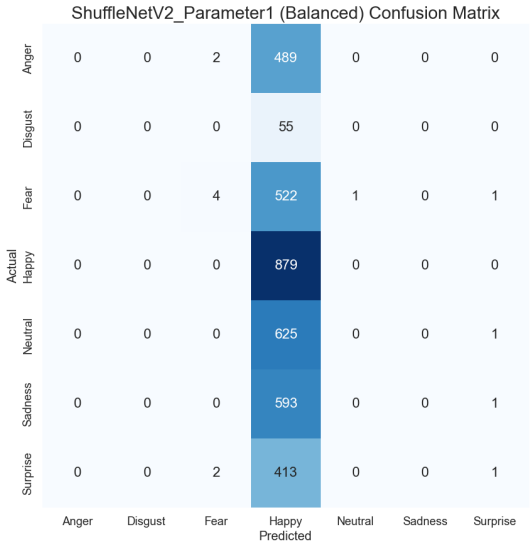


Figure 27 : Confusion matrix of ShuffleNetV2, Parameter 1

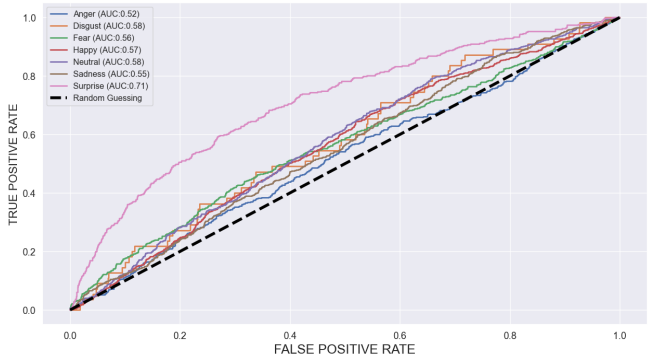


Figure 28 : ROC curve of ShuffleNetV2, Parameter 2

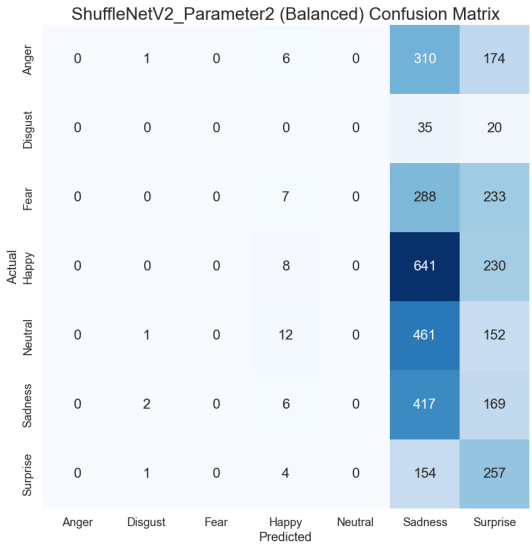


Figure 29 : Confusion matrix of ShuffleNetV2, Parameter 2

4.7. MobileNetV2

4.7.1. Training Imbalanced Dataset

In the first parameter, three fully connected (dense) layers are added to the Sequential model. These layers are used to perform the final classification of the facial expressions into 7 categories. The first dense layer has 512 neurons and uses a L2 regularisation term with a regularisation strength of 0.01 to prevent overfitting. The L2 regularisation term adds a penalty term to the loss function based on the magnitude of the weights in the layer. The BatchNormalization layer is used to normalise the activations of the dense layer, which can help to stabilise the training process. The activation function for the dense layer is a rectified linear unit (ReLU) activation function, which is defined using the Activation layer with an argument of 'relu'. The Dropout layer is used to randomly drop out a fraction of the activations during training, which can also help to prevent overfitting. The second dense layer has 256 neurons and uses a similar structure as the first dense layer, with a L2 regularisation term, BatchNormalization, a ReLU activation function, and a Dropout layer. Finally, the output layer has 7 neurons and uses a softmax activation function. The softmax activation function is used because the task involves multiclass classification, where the outputs represent class probabilities that must add up to 1.

Figure 30 shows that the accuracy results of the training set after fitting the architecture layers is 60.65% and the accuracy of the validation set is 55.13%. As can be seen in the figure, the training is gradually increasing over time. For validation is unstabilised at the first 6 epochs and starts to gradually increase after epoch 7.5. This shows that the model is learning well and not overfitting. Similar to the graph of accuracy, the model's loss for both training is steadily decreasing over time, even though there is slight fluctuation in epoch 5 but it still strongly indicates that the model is learning and increasing its performance on the validation set.

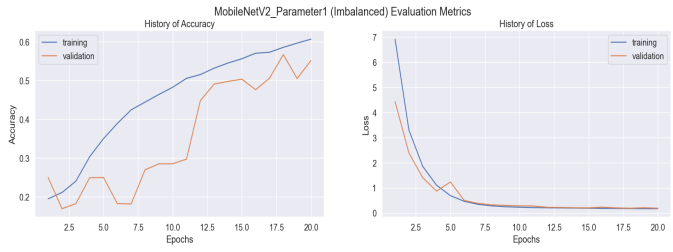


Figure 30 : Training and validation using MobileNetV2, Parameter 1 on Imbalanced Dataset

For second parameter architecture, the layers that were used in the second experiment of another model are being adopted and added to the sequential model. The accuracy of the training set is 53.96% and the accuracy of validation is 58.73%. Compared to the previous parameter, the accuracy of the train set is decreased but for validation is increased. This shows that the model is generalised fine with parameter 2 as the graph history of the accuracy train and valid set of the model is gradually increasing as shown in Figure 31. The loss for training is steadily decreasing but for the valid set there is fluctuation in between epoch 5 to 7.5 before continuing to decrease lower than the training set.

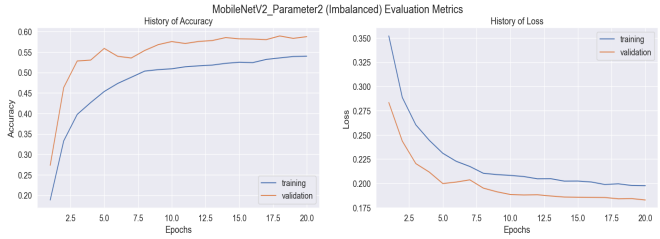


Figure 31 : Training and validation using MobileNetV2, Parameter 2 on Imbalanced Dataset

Upon implementing the data to both architecture parameters, the AUC-ROC score achieved by this model with the balance is 87.3%. The AUC-ROC results are the same for both architecture parameters.

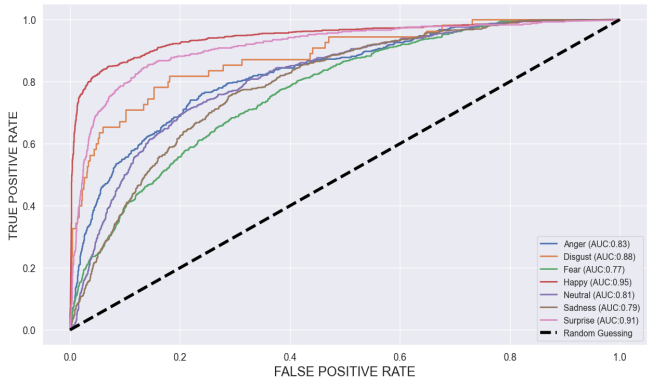


Figure 32 : ROC curve of ShuffleNetV2, Parameter 1

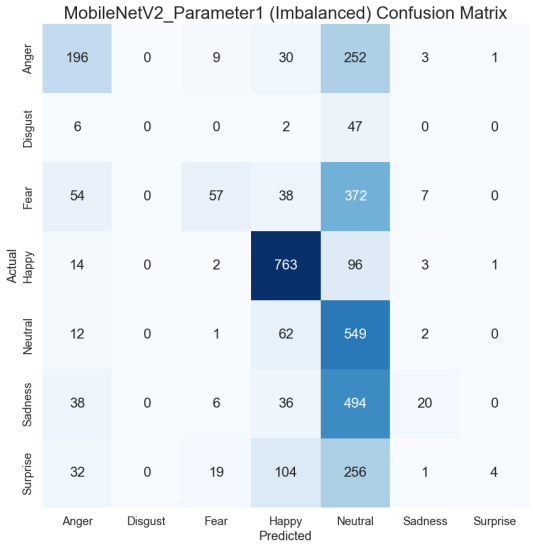


Figure 33 : Confusion matrix of MobileNetV2, Parameter 1

As can be seen in Figure 33, for the imbalanced dataset, the highest correct predictions are 'happy'. A significant number of 'neutral' samples are falsely predicted as other emotion classes even though the correct predicted is 549 for 'neutral'. 'Disgust', 'fear', 'sadness' and 'surprise' classes are not predicted well with a very low number of correct predictions.

4.7.2. Training Balanced Dataset

Figure 34 shows that the accuracy results of the training set after fitting the architecture layers is 73.3% and the accuracy of the validation set is 63.91%. As can be seen in the figure, the training is gradually increasing over time. For validation, at the first 6 epochs the accuracy gradually increases but after epoch 6, the accuracy is slightly constant around 63%. Similar to the graph of accuracy, the model's loss for training is steadily decreasing over time. But validation loss is decreasing gradually only at the first epoch 6 and starting to constant around 16.5% which indicates that the model is underfitting and the model is unable to accurately model the training data which causes large errors.

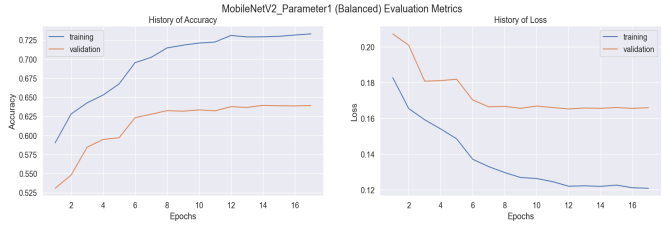


Figure 34 : Training and validation using MobileNetV2, Parameter 2 on Balanced Dataset

For the second parameter, the accuracy of the training set is 56.86% and the accuracy of validation is 62.03%. Compared to the previous parameter, the accuracy of the train set decreases significantly and the validation accuracy is slightly decreased. However, this model generalises well with the second parameter as the graph history of the accuracy train and valid set of the model is gradually increasing and the validation accuracy is much higher than training as shown in Figure 35. The loss for both training and validation also steadily decreases and the validation loss is much lower than the training set.

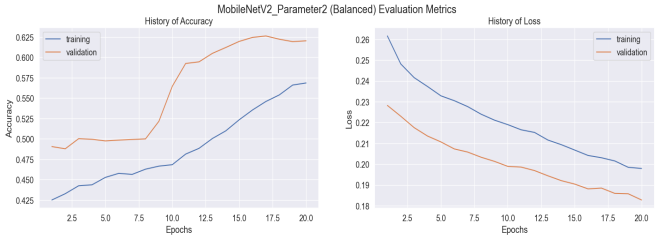


Figure 35 : Training and validation using MobileNetV2, Parameter 2 on Balanced Dataset

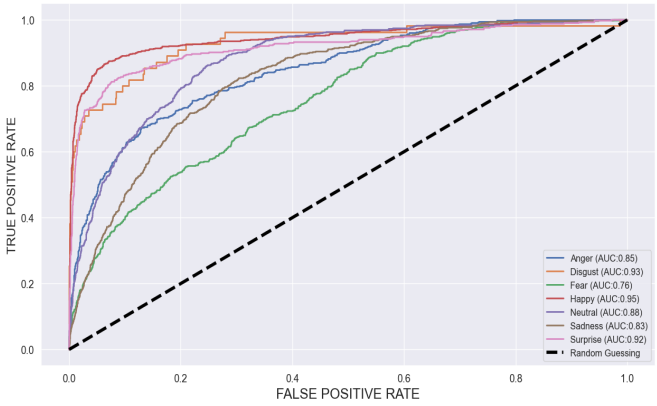


Figure 36 : ROC curve of MobileNetV2, Parameter 2

After implementing the data to both architecture parameters, the AUC-ROC score for the imbalanced dataset is 85%. When compared to the imbalanced dataset, this shows that the both parameter architecture models are better with a balanced dataset.

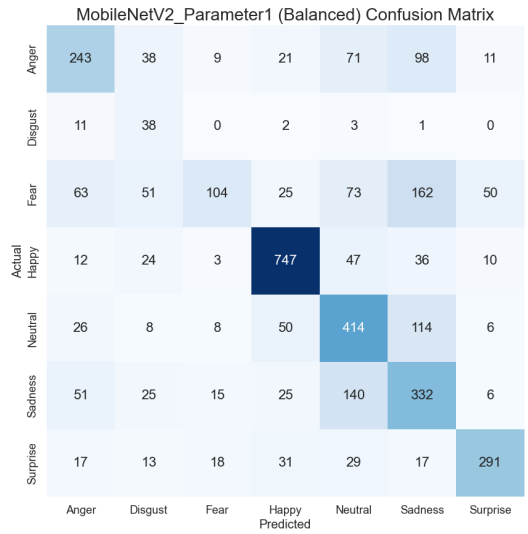


Figure 37 : Confusion matrix of MobileNetV2, Parameter 2

As can be seen in Figure 37, the most dominant class is 'happy' as most of the correct predictions for both experiments are happy with the highest number of correct predictions amongst 7 classes. Most of the classes are predicted well except for 'disgust' as the lowest is class disgust. The confusion matrix results are the same for both architecture parameters.

4.8. Overall Analysis

In table 1, a table on imbalanced dataset evaluation is shown. It is clear that DenseNet achieves the best accuracy on both parameters. The test accuracy also suggests that DenseNet201 has the best performance in terms of making correct predictions on a given dataset where it measures the number of correct predictions made by the model over the total number of predictions made. The model may have learned to identify patterns in the training data that enable it to make accurate predictions on new data.

Table 1 : Side-by-side evaluation on Imbalanced dataset

Model	Classifier Parameter	Train Accuracy	Test Accuracy	AUC-ROC Score
DenseNet-201	1	72%	64%	89%
	2	58%	61%	88%
Efficient Net-B0	1	34%	17%	47%
	2	25%	24%	51%
ShuffleNet V2	1	25%	27%	62%
	2	20%	19%	58%
MobileNet V2	1	61%	44%	85%
	2	54%	44%	85%

Following the results for a balanced dataset, the total results are shown in Table 2. Table 2 makes it clear that, when compared to the other models, DenseNet201 has the highest accuracy (75% for both parameters). DenseNet201 also performs best in terms of test accuracy for correctly making predictions on the provided dataset, followed by MobileNetV2. The model might have developed the ability to spot patterns in the training data that let it correctly predict outcomes from brand-new data.

Table 2 : Side-by-side evaluation on Balanced dataset

Model	Classifier Parameter	Train Accuracy	Test Accuracy	AUC-ROC Score
DenseNet-201	1	75%	64%	89%
	2	64%	61%	88%
Efficient Net-B0	1	35%	17%	47%
	2	15%	24%	51%
ShuffleNet V2	1	41%	25%	62%
	2	19%	19%	58%
MobileNet V2	1	73%	60%	87%
	2	57%	60%	87%

A sample of predictions, using DenseNet-201, is shown in the figure below.

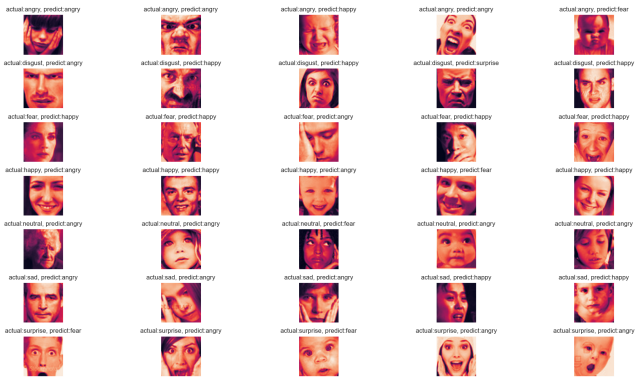


Figure 38 : Model prediction visualisation (DenseNet-201)

5. CONCLUSION

This research aims to adopt deep learning in recognizing emotion and investigate the potential use cases for CNN architectures. To boost the useful portions of photos while maintaining the original image resolution, we specifically improved the dataset by balancing it using the data augmentation approach. The effectiveness of architectures is improved by this strategy according to experimental results.

In summary, the project findings show that DenseNet201 has the best performance in recognizing emotion which obtains an accuracy of 75% on the balanced dataset and AUC value of 88% ± 1%, respectively. Hence, it may provide an efficient and reliable alternative for utilisation in relevant areas, especially in facial emotion recognition.

6. REFERENCES

[1] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely Connected Convolutional Networks," 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2261–2269, Jul. 2017, doi: 10.1109/cvpr.2017.243.

[2] S.-H. Wang and Y.-D. Zhang, "DenseNet-201-Based Deep Neural Network with Composite Learning Factor and Precomputation for Multiple Sclerosis Classification," ACM Transactions on Multimedia Computing, Communications, and Applications, vol. 16, no. 2s, pp. 1–19, Jul. 2020, doi: 10.1145/3341095.

[3] V. Agarwal, "Complete Architectural Details of all EfficientNet Models," Medium, May 31, 2020. https://towardsdatascience.com/complete-architectural-details-of-all-efficientnet-models-5fd5b736142

[4] J. Wang, Q. Liu, H. Xie, Z. Yang, and H. Zhou, "Boosted EfficientNet: Detection of Lymph Node Metastases in Breast Cancer Using Convolutional Neural Networks," Cancers, vol. 13, no. 4, p. 661, Feb. 2021, doi: 10.3390/cancers13040661.

[5] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design," arXiv:1807.11164 [cs.CV], Jul. 2018, [Online]. Available: https://arxiv.org/abs/1807.11164

[6] Y. Sari, Y. F. Arifin, Novitasari, and M. R. Faisal, "The Effect of Batch Size and Epoch on Performance of ShuffleNet-CNN Architecture for Vegetation Density Classification," 7th International Conference on Sustainable Information Engineering and Technology 2022, Nov. 2022, doi: 10.1145/3568231.3568239.

[7] Y. Li, X. Wang, B. Shi, and M. Zhu, "Hand Gesture Recognition Using IR-UWB Radar with ShuffleNet V2," Proceedings of the 5th International Conference on Control Engineering and Artificial Intelligence, Jan. 2021, doi: 10.1145/3448218.3448233.

[8] Z. Chen, J. Yang, L. Chen, and H. Jiao, "Garbage classification system based on improved ShuffleNet v2," Resources, Conservation and Recycling, vol. 178, p. 106090, Mar. 2022, doi: 10.1016/j.resconrec.2021.106090.

[9] A. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," 2017. [Online]. Available: https://arxiv.org/pdf/1704.04861.pdf

[10] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," 2019. [Online]. Available: https://arxiv.org/pdf/1801.04381.pdf

[11] K. Ali, Z. A. Shaikh, A. A. Khan, and A. A. Laghari, "Multiclass skin cancer classification using EfficientNets – a first step towards preventing skin cancer," Neuroscience Informatics, vol. 2, no. 4, p. 100034, Dec. 2022, doi: 10.1016/j.neuri.2021.100034.

[12] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," arXiv.org, 2019. https://arxiv.org/abs/1905.11946

[13] A. Gurumoorthy, "ImageNet," Devopedia, Jul. 03, 2019. https://devopedia.org/imagenet

[14] M. Ç. Aksoy, "EuroSAT with Keras - ShuffleNet," kaggle.com. https://www.kaggle.com/code/mcagriaksoy/eurosat-with-keras-shufflenet (accessed Feb. 02, 2023).

[15] "Challenges in Representation Learning: Facial Expression Recognition Challenge," kaggle.com. https://www.kaggle.com/competitions/challenges-in-representation-learning-facial-expression-recognition-challenge/data

[16] I. J. Goodfellow et al., "Challenges in Representation Learning: A Report on Three Machine Learning Contests," Neural Information Processing, pp. 117–124, 2013, doi: 10.1007/978-3-642-42051-1_16.

[17] A. Buslaev, V. I. Iglovikov, E. Khvedchenya, A. Parinov, M. Druzhinin, and A. A. Kalinin, "Albumentations: Fast and Flexible Image Augmentations," Information, vol. 11, no. 2, p. 125, Feb. 2020, doi: 10.3390/info11020125.

[18] S. Begaj, A. O. Topal, and M. Ali, "Emotion Recognition Based on Facial Expressions Using

Convolutional Neural Network (CNN),” IEEE Xplore, Dec. 01, 2020. https://ieeexplore.ieee.org/document/9302866/authors#authors (accessed Nov. 30, 2022).

[19] B. P. C, “Batch and Layer Normalization,” Pinecone. https://www.pinecone.io/learn/batch-layer-normalization/ (accessed Feb. 01, 2023).